Laboratorium nr 4 – wektoryzacja tekstu i klasyfikacja: spam albo sentyment

Na wykładzie 6 przeszliśmy drogę od tekstu do decyzji klasyfikatora: reprezentacje rzadkie (Bag-of-Words, TF-IDF), gęste (Word2Vec, Doc2Vec), podział danych, strojenie hiperparametrów i metryki. Na laboratorium porównacie kilka reprezentacji tekstu i kilka klasyfikatorów na tych samych danych.

Zasada jest prosta: **liczy się procedura, nie sam końcowy wynik**.

Organizacja pracy

Pracujecie w parach. Para wybiera wspólnie **zadanie** i **poziom trudności** (3.0 / 4.0 / 5.0):

- **zadanie A — detekcja spamu** albo
- **zadanie B — analiza sentymentu**.

Obowiązuje Python i biblioteki otwarte takie jak: scikit-learn, gensim, nltk, optuna

Poziomy są narastające: na 4.0 obowiązuje wszystko z poziomu 3.0 oraz wymagania dodatkowe, na 5.0 — wszystko z poziomu 4.0 oraz kolejne wymagania.

Poziomy w skrócie

Poziom	Reprezentacje tekstu	Klasyfikatory	Strojenie	Oddajecie
3.0	2	3	random search	kod + README z wynikami
4.0	4	5	random search + grid search	+ sprawozdanie PDF, odpowiedź ustna
5.0	6	7	random search + grid search + Optuna	+ sprawozdanie PDF, odpowiedź ustna

Każdy klasyfikator uruchamiacie na każdej reprezentacji. W wynikach dla każdego połączenia: **accuracy**, **F1-score** i **macierz pomyłek**.

Zbiory danych

Zadanie	Zbiór	Wielkość	Skąd
A — spam	SMS Spam Collection (Almeida i in., 2011)	5 574 SMS-y, 13,4% spamu	archive.ics.uci.edu/dataset/228 kopia: kaggle.com/datasets/uciml/sms-spam-collection-dataset licencja CC BY 4.0
B — sentyment	Sentence Polarity v1.0 (Pang i Lee, 2005)	10 662 zdania z recenzji filmów, 50% / 50%	cs.cornell.edu/people/pabo/movie-review-data (plik rt-polaritydata.tar.gz) albo w nltk: `nltk.download("sentence_polarity")`

Oba zbiory są małe i krótkie: pojedynczy model uczy się w sekundach. Najwolniejszy jest SVC z jądrem RBF na TF-IDF — ok. pół minuty na zbiorze B. Całe zadanie da się więc policzyć na domowym komputerze. Etykiety: spam = 1, zwykła wiadomość = 0; pozytywny = 1, negatywny = 0.

Można wybrać też dowolne inne zbiory danych w zakresie detekcji spamu czy analizy sentymentu.

Reprezentacje tekstu do wyboru

Grupa	Reprezentacja	Przykład w kodzie
rzadkie	Bag-of-Words	CountVectorizer(min_df=2)
rzadkie	TF-IDF słowny	TfidfVectorizer(ngram_range=(1, 2), min_df=2)
rzadkie	TF-IDF znakowy	TfidfVectorizer(analyzer="char_wb", ngram_range=(3, 5))
gęste	LSA	TF-IDF + TruncatedSVD(n_components=200)
gęste	Word2Vec + średnia	Word2Vec(...) i średnia wektorów słów
gęste	Word2Vec + średnia ważona TF-IDF	jak wyżej, wagi z TfidfVectorizer
gęste	Doc2Vec	Doc2Vec(dm=0) albo Doc2Vec(dm=1), infer_vector
gęste	FastText + średnia	gensim.models.FastText(...)
gotowe	GloVe + średnia	gensim.downloader: "glove-wiki-gigaword-100"
gotowe	fastText + średnia	gensim.downloader: "fasttext-wiki-news-subwords-300"

- **3.0:** 2 reprezentacje — jedna rzadka i jedna gęsta, np. TF-IDF słowny i Doc2Vec;
- **4.0:** 4 reprezentacje — co najmniej 2 rzadkie i 2 gęste, w tym Word2Vec, np. Bag-of-Words, TF-IDF słowny, Word2Vec + średnia, Doc2Vec;
- **5.0:** 6 reprezentacji — jak na 4.0 oraz co najmniej jedna gotowa (GloVe albo fastText), np. dodatkowo TF-IDF znakowy i GloVe + średnia;
- każda reprezentacja to inny wiersz tabeli — dwa warianty tej samej metody (np. dm=0 i dm=1) liczą się jako jedna.

Klasyfikatory do wyboru

Naive Bayes (`MultinomialNB` lub `ComplementNB` dla reprezentacji rzadkich, `GaussianNB` dla gęstych), `LogisticRegression`, `LinearSVC`, `SVC` (jądro RBF), `KNeighborsClassifier`, `DecisionTreeClassifier`, `RandomForestClassifier`, `ExtraTreesClassifier`, `HistGradientBoostingClassifier` (tylko gęste), `AdaBoostClassifier`, `SGDClassifier`, `RidgeClassifier` oraz dowolne inne np. z https://scikit-learn.org/stable/supervised_learning.html

Zadanie na 3.0

- wczytanie zbioru, lowercase, usunięcie stopwords i lematyzacja;
- podział zbioru danych na uczący i testowe
- **2 reprezentacje** i **3 klasyfikatory**, każdy klasyfikator na każdej reprezentacji;
- dobór hiperparametrów przez **random search**
- wyniki na zbiorze testowym: tabela **accuracy** i **F1-score** dla każdego połączenia oraz **macierze pomyłek**.

Poziom 3.0 nie wymaga sprawozdania ani odpowiedzi ustnej — wystarczy README z instrukcją uruchomienia, tabelą wyników i macierzami pomyłek (PNG).

Zadanie na 4.0

- wszystko z poziomu 3.0;
- **4 reprezentacje** (w tym Word2Vec ze średnią albo średnią ważoną) i **5 klasyfikatorów**;
- dobór hiperparametrów dwiema metodami: **random search** i **grid search**

Ponadto: sprawozdanie PDF oraz odpowiedź ustna.

Zadanie na 5.0

- wszystko z poziomu 4.0;
- **6 reprezentacji** (w tym co najmniej jedna gotowa) i **7 klasyfikatorów**;
- trzecia metoda strojenia: **Optuna** (np. 30–50 prób) — porównanie random search, grid search i Optuny

Ponadto: sprawozdanie PDF oraz odpowiedź ustna.

Sprawozdanie (poziomy 4.0 i 5.0)

- krótki opis zbioru i preprocessingu;
- opis reprezentacji (z parametrami) i zakresów hiperparametrów;
- tabela wyników: reprezentacja × klasyfikator × metoda strojenia — accuracy i F1-score;
- macierze pomyłek;
- wnioski: która reprezentacja i który klasyfikator sprawdziły się najlepiej.

Oddanie pracy

Na platformie Delta wgrajcie jedno archiwum na parę, o nazwie:

LAB4_Nazwisko_Imie_Nazwisko_Imie.zip

Archiwum zawiera kod w Pythonie, README z instrukcją uruchomienia i informacją, które zadanie wybraliście, oraz macierze pomyłek w PNG. Na poziomach 4.0 i 5.0 dochodzi sprawozdanie PDF. Zbioru danych nie dołączajcie.